

Multimodal AI : How LLMs Understand Images

Coder Army

Why Text-Only AI Is Not Enough

Most basic LLM applications follow this flow:

```
User
↓
Text Prompt
↓
Application
↓
LLM
↓
Text Response
```

For example:

```
"Summarize this support ticket."
```

The application sends text to the model, and the model returns text.

Conceptually:

```
String → LLM → String
```

This works well when all useful information is available as text.

But real-world information does not always come in text form.

A user may say:

```
"My payment failed. See attached screenshot."
```

In this case, the important information may exist inside:

payment-error.png

Similarly, users may upload:

invoice.jpg
receipt.png
dashboard-screenshot.png
error-screenshot.png
product-image.jpg

A text-only AI application cannot directly understand the information contained inside these images.

This is where multimodal AI becomes useful.

What Is Multimodality?

A **modality** is simply a form in which information exists.

Common modalities include:

Text
Image
Audio
Video

A traditional text model works like:

Text
↓
Model
↓
Text

A multimodal model can work with more than one type of information.

For example:

Text + Image

↓

Model

↓

Text

In this lecture, the focus is specifically on:

Text + Image → Model → Text

Multimodal vs Multi-Model

These two terms are different.

Multi-model

Using multiple different AI models.

For example:

GPT
+
Claude
+
Gemini

Multimodal

Using multiple forms of information with a model.

For example:

Text
+
Image

So an application that sends a question along with an image is a **multimodal AI application**.

A Simple Example

Suppose a user uploads an invoice:

```
-----  
ABC Electronics  
  
Invoice No: INV-3821  
Date: 07 Oct 2026  
  
MacBook Air      ₹82,000  
  
Total            ₹82,000  
-----
```

A traditional application may only know that a file called:

```
invoice.png
```

was uploaded.

A multimodal model can receive the image along with an instruction:

```
Extract:  
  
- Invoice number  
- Vendor  
- Date  
- Total amount
```

And return:

```
{  
  "invoiceNumber": "INV-3821",  
  "vendor": "ABC Electronics",  
  "date": "07 Oct 2026",  
  "amount": 82000  
}
```

The important change is:

The image itself has now become part of the model input.

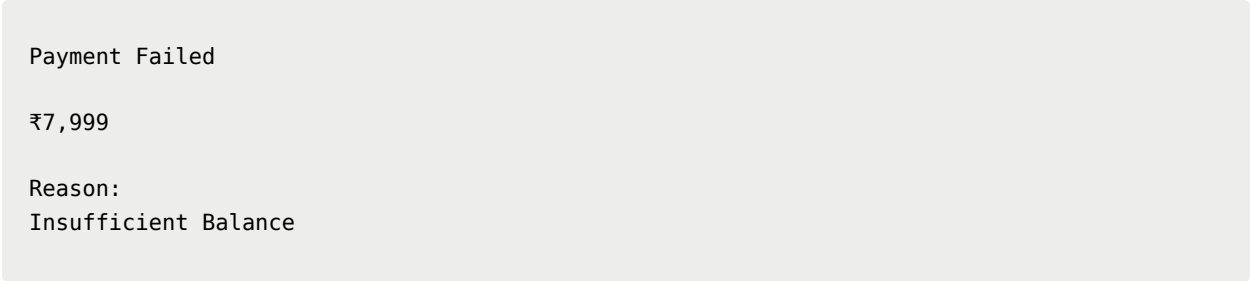
What Can a Multimodal Model Understand?

A multimodal model does more than simply read text from an image.

It may understand:

- Objects
- Text
- Layout
- Position
- Relationships
- Context
- Meaning

Consider this screenshot:



Payment Failed

₹7,999

Reason:
Insufficient Balance

If we ask:

What happened?

The model may answer:

The payment failed because the account had insufficient balance.

This is different from simply extracting the characters visible in the screenshot.

OCR vs Multimodal LLM

OCR

OCR stands for:

Optical Character Recognition

OCR existed long before modern LLMs.

Its primary job is:

```
Image
↓
Recognize Characters
↓
Extract Text
```

For example, an image containing:

```
Amazon
Order #82931
Total: ₹4,999
```

may produce:

```
Amazon
Order #82931
Total ₹4,999
```

OCR is mainly concerned with:

```
Image → Text
```

Multimodal LLM

A multimodal model can go beyond character extraction.

Suppose we ask:

```
"What is the order number and total amount?"
```

The model may answer:

```
Order Number: 82931  
Total Amount: ₹4,999
```

We could also ask:

```
"Has this order been paid successfully?"
```

The model can examine the image and reason about its meaning.

Conceptually:

OCR

```
Image  
↓  
Text
```

Multimodal Model

```
Image + Question  
↓  
Understanding  
↓  
Answer
```

A useful intuition is:

| OCR is like asking someone to read everything written on a page.

A multimodal LLM is closer to asking:

| Look at this page and tell me what is important.

How Does an LLM Understand an Image?

To understand images, start with something we already know about text.

How Text Reaches an LLM

Suppose we send:

```
"Why is my payment failing?"
```

The model does not directly process the English sentence.

First, a tokenizer converts the text into tokens.

Conceptually:

```
"Why"  
"is"  
"my"  
"payment"  
"failing"  
"?"
```

The actual tokenization may be different, but we can think of it as:

```
Token 1  
Token 2  
Token 3  
Token 4  
...
```

Each token is eventually represented numerically.

Token
↓
Embedding
↓
Vector

Therefore, an LLM does not literally work with English words.

It ultimately processes:

numbers
and
vectors

This is the key idea needed to understand image processing.

Images Are Also Numbers

An image may look visual to us, but to a computer it is numerical data.

A normal RGB image contains pixels.

Each pixel contains values for:

Red
Green
Blue

For example:

R = 255
G = 120
B = 30

A 1920×1080 image contains more than two million pixels.

Conceptually, the raw image is a large numerical grid:

```
[
  [pixel, pixel, pixel, ...],
  [pixel, pixel, pixel, ...],
  [pixel, pixel, pixel, ...]
]
```

However, feeding millions of raw pixel values directly into a language model would be inefficient.

So images are processed differently.

Step 1 — Divide the Image Into Patches

A common approach used by modern vision models is to divide an image into smaller regions called **patches**.

Imagine an image:

```
+-----+
|           |
|    CAT    |
|           |
|           |
+-----+
```

It can be divided into smaller sections:

```
+---+---+---+---+
| P1 | P2 | P3 | P4 |
+---+---+---+---+
| P5 | P6 | P7 | P8 |
+---+---+---+---+
| P9 | P10 | P11 | P12 |
+---+---+---+---+
```

Each section is an image patch.

Conceptually:

```

Image
↓
Split Into Patches
↓
Patch 1
Patch 2
Patch 3
...

```

The exact patch size depends on the architecture.

A patch might, for example, represent a small region such as:

```

16 × 16 pixels

```

The important idea is not the exact size.

The important idea is:

Instead of processing the entire image as one giant object, the model breaks it into smaller visual units.

Text Tokens vs Image Patches

This gives us a useful analogy.

Text

```

Sentence
↓
Tokens
↓
Token Embeddings

```

Image

```

Image
↓

```

Patches
↓
Visual Embeddings

Image patches are not exactly the same thing as text tokens, but they play a similar conceptual role.

Step 2 — Vision Encoder

An image patch still contains raw pixel values.

The LLM cannot directly reason over those pixels.

So the patches are processed by another neural network, commonly referred to conceptually as a:

Vision Encoder

Flow:

Image
↓
Patches
↓
Vision Encoder
↓
Visual Embeddings

The vision encoder converts image information into numerical vectors.

For example:

Patch Pixels
↓
Vision Encoder
↓
[0.12, -0.55, 1.04, 0.31, ...]

This vector is called a **visual embedding** or visual representation.

It is not human-readable.

It is a learned numerical representation of the visual information.

Text Embeddings and Visual Embeddings

For text:

```
"cat"  
↓  
Tokenizer  
↓  
Token  
↓  
Embedding  
↓  
[0.31, -0.18, 0.77, ...]
```

For an image:

```
Image region containing a cat's ear  
↓  
Vision Encoder  
↓  
Visual Embedding  
↓  
[0.28, -0.11, 0.81, ...]
```

Eventually both text and image information become:

```
vectors
```

This is how the two worlds can eventually interact.

Step 3 — Connecting Vision to the Language Model

There is still one problem.

The visual embeddings produced by the vision encoder may not be directly compatible with the representation expected by the language model.

So multimodal architectures commonly contain some form of:

Projection Layer
or
Adapter

Conceptually:

```
Image
↓
Vision Encoder
↓
Visual Embeddings
↓
Projection / Adapter
↓
Representation usable by the LLM
```

Its job is essentially:

Transform the visual representation into a form that the language model can work with.

Why Position Matters in an Image

Images are not just collections of objects.

Their location matters.

For example:

Button below text box

has a different meaning from:

Button above text box

Similarly:

Error message next to payment field

contains important structural information.

So visual models also need to understand:

Patch Content
+
Patch Position

This helps the model understand the layout and relationships between objects.

How Does the Model Recognize an Object?

Suppose an image contains a cat.

Different patches may contain:

fur
ear
eye
whiskers
background

One patch alone may not be enough to identify a cat.

But the model can understand relationships across patches.

Conceptually:

Patch 12 → ear
Patch 13 → eye

Patch 14 → fur
Patch 22 → whiskers

Together:

ear + eye + fur + whiskers
↓
cat

This is similar to how context works in language.

The word:

bank

alone is ambiguous.

But:

I deposited money in the bank.

provides enough surrounding information to understand the intended meaning.

Similarly, one image patch may not contain enough information, but multiple related patches together form meaningful objects.

Vision Transformers and Attention

Transformers are not limited to text.

The same general attention idea can also be applied to image patches.

For text:

Word Token
Word Token
Word Token
↓
Self-Attention

↓
Relationships Between Words

For images:

Patch
Patch
Patch
↓
Attention
↓
Relationships Between Image Regions

Instead of asking:

Which words are related?

the vision model can learn:

Which visual regions are related?

This allows the model to combine small pieces of visual information into larger meaningful concepts.

Reading Text Inside Screenshots

Suppose a screenshot contains:

Insufficient Balance

Traditional OCR explicitly follows:

Pixels
↓
Character Recognition

↓
Text

Modern multimodal models can also learn visual patterns representing:

- Letters
- Words
- Numbers
- Layout
- UI elements

Conceptually:

```
Screenshot Pixels
↓
Visual Representation
↓
Recognize Written Content
↓
Language Reasoning
```

This means we do not necessarily need to add a separate OCR system before sending a screenshot to a multimodal model.

How Does the Model Learn That a Cat Is a Cat?

Multimodal models are trained using large amounts of visual and language information.

Conceptually, training data may contain examples such as:

```
[Image of a cat]

"This is a cat."
```

or:

[Image]

Question:

What animal is shown?

Answer:

Cat

Across huge amounts of training data, the model learns relationships between:

Visual Patterns

↑

Language Concepts

For example:

ears
fur
whiskers
face shape

become associated with concepts such as:

cat
kitten
animal
pet

This connection between visual information and language is often described broadly as **vision-language alignment**.

Combining Text and Image

Now consider an actual request:

"Why is this payment failing?"

+

payment.png

Two processing pipelines are involved.

Text Side

"Why is this payment failing?"

↓

Tokenizer

↓

Text Tokens

↓

Text Embeddings

Image Side

payment.png

↓

Image Preprocessing

↓

Image Patches

↓

Vision Encoder

↓

Visual Embeddings

↓

Projection / Adapter

The two representations are then brought together:

Text Representations

+

Visual Representations

↓

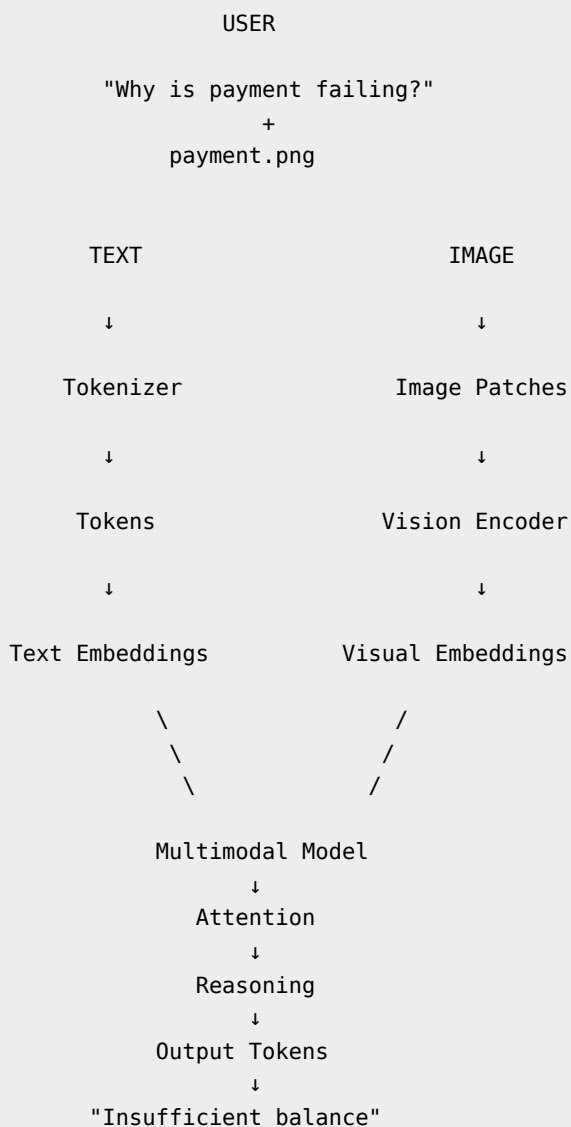
Multimodal Model

↓
Reasoning
↓
Response

This is the central idea behind multimodal understanding.

End-to-End Flow

A simplified architecture looks like this:



The most important idea is:

An LLM does not really work with English words or JPEG files. It works with numerical representations.

Text becomes:

Token Embeddings

Images become:

Visual Embeddings

The multimodal model is trained to reason across both.

Why Large Images May Be Preprocessed

Suppose someone uploads:

8000 × 6000 image

Processing every pixel at full resolution would be expensive.

So before the vision encoder receives the image, the provider may perform preprocessing such as:

Resize
Crop
Tile
Normalize

Conceptually:

Original Image
↓
Resize / Tile

↓
Manageable Representation
↓
Vision Encoder

Very large images may also be divided into multiple regions so smaller details can still be preserved.

The exact implementation depends on the model and provider.

Image Quality Still Matters

Multimodal models are powerful, but the quality of the input still affects the result.

Models may struggle with:

- Extremely small text
- Blurry screenshots
- Low-resolution images
- Poor lighting
- Cropped information
- Very dense screenshots
- Low contrast

If the visual information is unclear, the model has less reliable information to work with.

How This Connects to Our Application

From our application's point of view, we simply send:

User Message

└─ Text
└─ Image

For example:

```
Text:
"Why is my payment failing?"

Image:
payment.png
```

The application or framework does not itself understand the image.

It simply packages the request and sends it to a multimodal model.

Conceptually:

```
Browser
↓
Application
↓
Text + Image
↓
Model Provider
↓
Multimodal Model
↓
Response
```

Under the hood:

```
Text
↓
Tokenizer
↓
Text Representations
```

and:

```
Image
↓
Vision Processing
↓
Visual Representations
```

Both are then used by the model to generate an answer.

Conversation History With Images

Images can also become part of conversation context.

For example:

USER:
[Payment Screenshot]

Why is payment failing?

Assistant:

The screenshot shows that the payment failed
because of insufficient balance.

Then the user asks:

How can I fix it?

If the previous user message, including the image, is sent again as part of conversation history, the model can continue reasoning about that same screenshot.

Conceptually:

Conversation History

USER:
Image + Question

ASSISTANT:
Answer

USER:
Follow-up Question

The multimodal model receives the relevant previous context again and can continue the conversation.

Key Mental Model

The entire process can be simplified into seven steps.

1. Text is not directly understood by the model

Text → Tokens

2. Tokens become numerical representations

Tokens → Embeddings

3. Images are also numerical data

Image → Pixels

4. Images are broken into manageable visual regions

Image → Patches

5. A vision encoder converts those regions into vectors

Patches → Visual Embeddings

6. Text and visual representations are processed together

Text Embeddings
+
Visual Embeddings

↓
Multimodal Model

7. The response is generated as normal text tokens

Model
↓
Output Tokens
↓
Text Response

Final Takeaway

A multimodal application does not fundamentally change how an AI application works.

Previously:

Text
↓
LLM
↓
Response

Now:

Text + Image
↓
Multimodal LLM
↓
Response

The important change happens inside the model pipeline.

Text is converted into:

Token Embeddings

Images are converted into:

Visual Embeddings

Once both forms of information are represented numerically, the multimodal model can reason across them together.

So the core idea is:

Text becomes tokens. Images become visual representations. A multimodal model learns how to connect both.